TEXT CLUSTERING USING K-MEANS

Python NLP Laboratory — Step-by-step execution and explanation

Objective: Implement K-Means clustering to group similar text documents into clusters without using predefined category labels.

1. What is Text Clustering?

Text clustering is the process of grouping similar text documents together based on their content. For example, documents about artificial intelligence, machine learning and Python may be grouped together, while documents about cricket and football may form another group. The algorithm discovers similarities from document content; it is not given the category labels.

Illustrative grouping: one cluster may contain technology-related documents, another sports-related documents, and another education-related documents. These names are assigned by a person after inspecting the clusters; K-Means itself outputs cluster numbers.

2. What is K-Means Clustering?

K-Means is an unsupervised Machine Learning algorithm that divides data into K groups called clusters. K represents the number of clusters. The algorithm represents each cluster by a centre called a centroid and assigns documents to clusters according to their distance from these centroids.

Important: We specify the number of clusters, but we do not specify which document belongs to which category.

3. How does K-Means work?

The workflow is: input text documents → text preprocessing → TF-IDF feature extraction → K-Means algorithm → final clusters (for example, Cluster 0, Cluster 1 and Cluster 2).

Step 1 — Select the number of clusters: Specify K = 3 if the algorithm should create three clusters.

Step 2 — Initialize centroids: The algorithm selects three initial cluster centres.

Step 3 — Assign documents: Each document is assigned to the cluster whose centroid is closest to it.

Step 4 — Update centroids: New centroids are calculated from the documents assigned to each cluster. Assignment and update steps repeat until the clusters stabilize or a stopping condition is reached.

4. What is TF-IDF and why is it used?

K-Means cannot directly process sentences. It requires numerical input. TF-IDF (Term Frequency–Inverse Document Frequency) converts text documents into numerical vectors and assigns weights to words based on their frequency in a document and across the document collection. K-Means uses these vectors to calculate distances and group similar documents.

5. Install the required Python libraries

Open Command Prompt or the VS Code terminal and execute:

```
pip install scikit-learn pandas
```

The program uses TfidfVectorizer to convert text into numerical features, KMeans to perform clustering, pandas to display results in a table, and Counter to count documents in each cluster.

6. Complete Python program

Create a Python file named **text_clustering_kmeans.py** and enter the following code:

```
# TEXT CLUSTERING USING K-MEANS
```

```
# STEP 1: Import required libraries
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.cluster import KMeans
import pandas as pd
from collections import Counter

# STEP 2: Create sample text documents
documents = [
    # Technology-related documents
    "Artificial intelligence is transforming healthcare",
    "Machine learning is used in data science",
    "Python programming is useful for software development",
    "Deep learning is a branch of artificial intelligence",
    "Computer networks connect different devices",

    # Sports-related documents
    "India won the cricket match",
    "Football is a popular sport",
    "The cricket team scored a century",
    "The football player scored a goal",
    "The team won the basketball tournament",

    # Education-related documents
    "Students attend classes in college",
    "Teachers explain programming concepts",
    "University conducts semester examinations",
    "Students prepare assignments and projects",
    "Online education provides learning opportunities"
]

# STEP 3: Convert text into TF-IDF features
vectorizer = TfidfVectorizer(stop_words="english")
X = vectorizer.fit_transform(documents)

print("TF-IDF FEATURE MATRIX:")
print(X.toarray())

# STEP 4: Apply K-Means clustering
kmeans = KMeans(
    n_clusters=3,
    random_state=42,
    n_init=10
)
kmeans.fit(X)

# STEP 5: Get cluster assignments
cluster_labels = kmeans.labels_

# STEP 6: Display documents and their clusters
print("\n===== DOCUMENT CLUSTERING RESULTS =====")
results = pd.DataFrame({
    "Document": documents,
    "Cluster": cluster_labels
})
print(results.to_string(index=False))

# STEP 7: Count documents in each cluster
print("\n===== NUMBER OF DOCUMENTS IN EACH CLUSTER =====")
cluster_counts = Counter(cluster_labels)
for cluster, count in sorted(cluster_counts.items()):
    print("Cluster", cluster, ":", count, "documents")

# STEP 8: Display important words in each cluster
print("\n===== IMPORTANT WORDS IN EACH CLUSTER =====")
terms = vectorizer.get_feature_names_out()
for i, center in enumerate(kmeans.cluster_centers_):
    top_indices = center.argsort()[-5:][::-1]
    top_words = [terms[index] for index in top_indices]
    print("\nCluster", i)
    print("Important words:", top_words)

# STEP 9: Predict cluster for a new document
new_documents = [
    "Artificial intelligence and machine learning",
    "India won the football tournament",
    "Students attend university classes"
]
new_features = vectorizer.transform(new_documents)
predicted_clusters = kmeans.predict(new_features)

print("\n===== NEW DOCUMENT CLUSTERING =====")
for document, cluster in zip(new_documents, predicted_clusters):
    print("\nDocument:", document)
    print("Assigned Cluster:", cluster)
```

7. Explanation of the Python program

Step 1 — Import the libraries

TfidfVectorizer converts text into numerical TF-IDF features. KMeans performs the clustering. pandas displays the documents and cluster assignments in a table. Counter counts how many documents are assigned to each cluster.

Step 2 — Create sample text documents

The program contains 15 documents covering three themes: Technology, Sports and Education. The comments in the code help a reader understand the sample dataset, but they are not category labels supplied to K-Means. The algorithm receives only the document text.

Step 3 — Convert text into TF-IDF features

```
vectorizer = TfidfVectorizer(stop_words="english")
X = vectorizer.fit_transform(documents)
```

TfidfVectorizer builds a vocabulary and converts each document into a numerical vector. stop_words='english' removes common English words. fit_transform() learns the vocabulary and calculates TF-IDF features for all supplied documents. Each row of the resulting matrix represents one document and each column represents a vocabulary term.

The statement X.toarray() displays the numerical feature matrix. In practice, TF-IDF often produces a sparse matrix, so displaying it as an array is useful only for this small teaching dataset.

Step 4 — Apply K-Means clustering

```
kmeans = KMeans(
    n_clusters=3,
    random_state=42,
    n_init=10
)
kmeans.fit(X)
```

n_clusters=3 asks the algorithm to create three clusters. random_state=42 makes the initialization reproducible. n_init=10 runs the initialization process ten times and retains the result with the lowest inertia. fit(X) performs the clustering using the TF-IDF matrix.

Step 5 — Understand the clustering process

K-Means initializes centroids, assigns each document to its nearest centroid, recalculates centroids using the assigned documents, and repeats the assignment and update process until convergence or a stopping limit. No predefined category labels are used.

Step 6 — Get cluster assignments

```
cluster_labels = kmeans.labels_
```

This retrieves the cluster number assigned to each document. For example, a document may be assigned to Cluster 0, Cluster 1 or Cluster 2. The numbers are identifiers only; Cluster 0 does not automatically mean Technology or Sports.

Step 7 — Display documents and clusters

```
results = pd.DataFrame({
    "Document": documents,
    "Cluster": cluster_labels
})
print(results.to_string(index=False))
```

This creates a table containing each original document and its assigned cluster number.

Step 8 — Count documents in each cluster

```
cluster_counts = Counter(cluster_labels)
for cluster, count in sorted(cluster_counts.items()):
    print("Cluster", cluster, ":", count, "documents")
```

Counter counts the number of documents assigned to each cluster. The counts are produced by the actual run and are not necessarily equal across clusters.

Step 9 — Identify important words in each cluster

```
terms = vectorizer.get_feature_names_out()
for i, center in enumerate(kmeans.cluster_centers_):
```

```
top_indices = center.argsort()[-5:][::-1]
top_words = [terms[index] for index in top_indices]
```

The centroid contains a weight for each vocabulary term. The code finds the five terms with the largest centroid weights and prints them. These terms can help a person interpret the theme of each cluster. The interpretation is not a category label learned from training.

Step 10 — Predict the cluster of new documents

```
new_features = vectorizer.transform(new_documents)
predicted_clusters = kmeans.predict(new_features)
```

The new sentences are transformed using the vocabulary and IDF weights already learned from the original documents. kmeans.predict() assigns each new vector to the nearest existing centroid. It does not create a new cluster.

8. Expected output

The program displays the TF-IDF feature matrix, document-to-cluster assignments, number of documents in each cluster, important words in each cluster, and cluster assignments for three new documents. Cluster numbers, counts and top words depend on the actual run.

```
===== DOCUMENT CLUSTERING RESULTS =====
Document                                    Cluster
Artificial intelligence is transforming...     [cluster]
Machine learning is used in data science        [cluster]
India won the cricket match                     [cluster]
Football is a popular sport                     [cluster]
Students attend classes in college               [cluster]

===== NUMBER OF DOCUMENTS IN EACH CLUSTER =====
Cluster 0 : [calculated count] documents
Cluster 1 : [calculated count] documents
Cluster 2 : [calculated count] documents

===== IMPORTANT WORDS IN EACH CLUSTER =====
Cluster 0
Important words: [top terms]

===== NEW DOCUMENT CLUSTERING =====
Document: Artificial intelligence and machine learning
Assigned Cluster: [cluster]
```

9. How to execute the program

1. Open IDLE, VS Code, PyCharm or another Python environment.
2. Install the libraries using `pip install scikit-learn pandas`.
3. Create a Python file named **text_clustering_kmeans.py**.
4. Paste the complete program and save it.
5. Open a terminal in the folder containing the file and run:

```
python text_clustering_kmeans.py
```

> No external dataset download is required because the sample documents are included in the program.

10. Difference between classification and clustering

Classification	Clustering
Supervised learning.	Unsupervised learning.
Requires predefined labels.	Does not require predefined labels.
Learns from labelled examples.	Discovers patterns automatically.
Predicts known categories.	Groups similar documents.
Examples: SVM and Naive Bayes.	Example: K-Means.

11. Applications of text clustering

• Grouping similar news articles
• Organizing research papers
• Grouping customer reviews
• Document management
• Grouping similar emails
• Topic discovery in social media
• Grouping research publications by subject

12. Student exercises

1. Increase the number of clusters from 3 to 4.
2. Add 10 more documents.
3. Create clusters for medical research articles.
4. Change TF-IDF to Bag-of-Words representation.
5. Identify the important words in each cluster.
6. Compare K-Means clustering with hierarchical clustering.

13. Quick knowledge check

Question 1: Is K-Means supervised or unsupervised learning?
Answer: Unsupervised learning.

Question 2: What does K represent in K-Means?
Answer: The number of clusters.

Question 3: Why is TF-IDF used in this experiment?
Answer: To convert text into numerical vectors.

Expected laboratory outcome: Students should be able to convert text into TF-IDF vectors, implement K-Means clustering, group similar documents without predefined labels, identify important words in each cluster and assign new documents to existing clusters.

Teaching suggestion: First explain the difference between supervised and unsupervised learning. Then demonstrate how three documents are converted into numerical vectors. Finally, execute K-Means and explain how the algorithm automatically groups similar documents.

Important note: Cluster numbers do not have predefined meanings. The algorithm may assign Technology to Cluster 0 in one run and Cluster 2 in another. Interpret cluster meaning by examining the documents and important words.